



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Impacto das Decisões de Design na Qualidade

Como decisões arquiteturais
tomadas no início do projeto
impactam a qualidade atual?



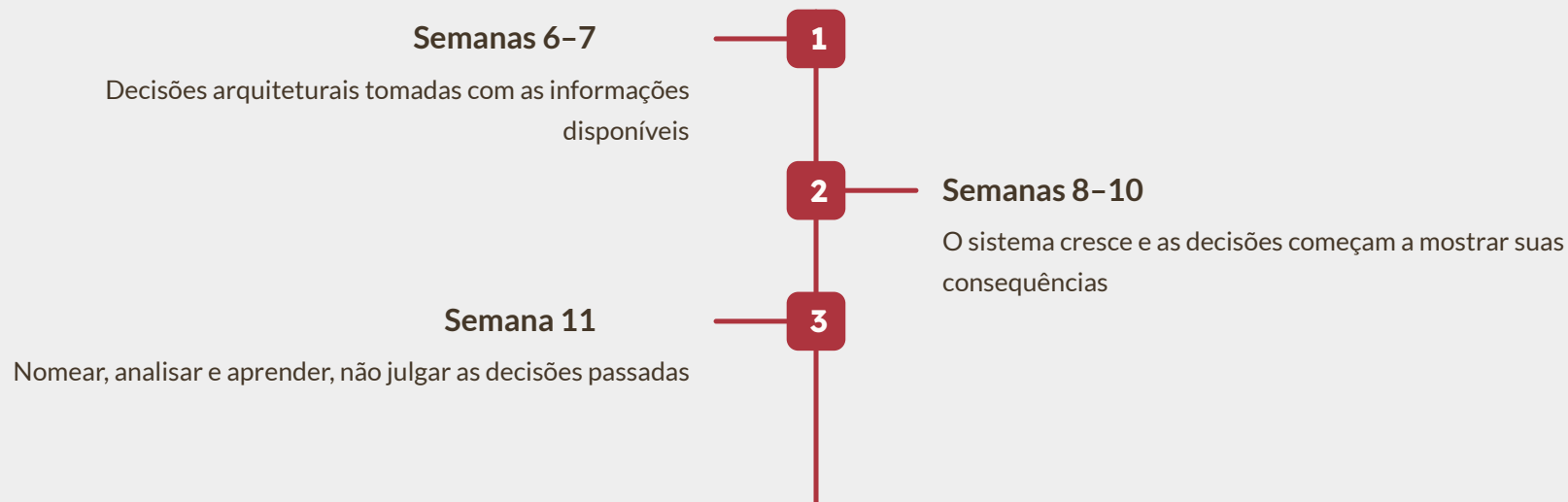


**"Aponte uma
decisão arquitetural
que sua equipe
tomou nas Semanas
6 ou 7 e que hoje
(Semana 11) está
cobrando o preço."**



O Momento da Colheita

Chegamos ao ponto da disciplina onde as escolhas do passado se tornam visíveis. Sprint 3 é o sprint de **qualidade e refactoring**, não de novas features.



Arquitetura É Decisão Duradoura



"Cada decisão arquitetural tem um impacto direto e duradouro sobre o sistema."

— Engenharia de Software em Dimensões, Cap. 14

Uma decisão é considerada **arquitetural** quando reúne três características simultâneas: é estruturalmente importante, é difícil de reverter e seu impacto se propaga ao longo do tempo.

✓ Arquitetural

"Construir como microsserviços comunicando via REST"

✗ Não arquitetural

"Usar Axios em vez de fetch no frontend"



A Arquitetura Afeta 4 Atributos de Qualidade

Ignorar atributos de qualidade durante as decisões arquiteturais não os elimina — apenas adia o custo de lidar com eles, geralmente em um momento muito pior.

Atributo	Sintoma quando ignorado
Manutenibilidade	"Se eu mexo aqui, quebra lá"
Desempenho	Lentidão crescente à medida que o sistema cresce
Segurança	Autenticação adicionada como remendo, não por design
Testabilidade	Impossível escrever testes unitários sem subir o banco inteiro

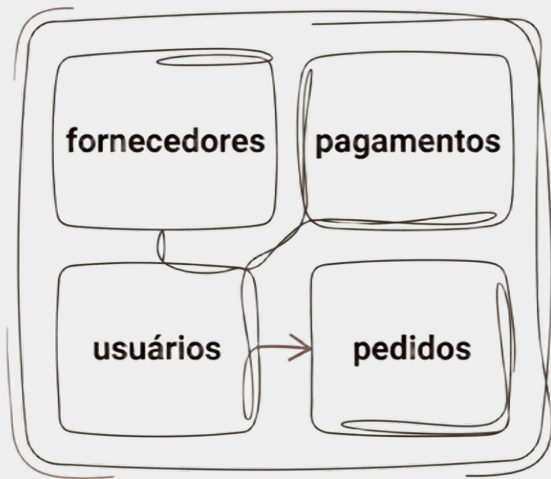
❓ Qual desses atributos está sendo mais impactado no seu projeto agora?



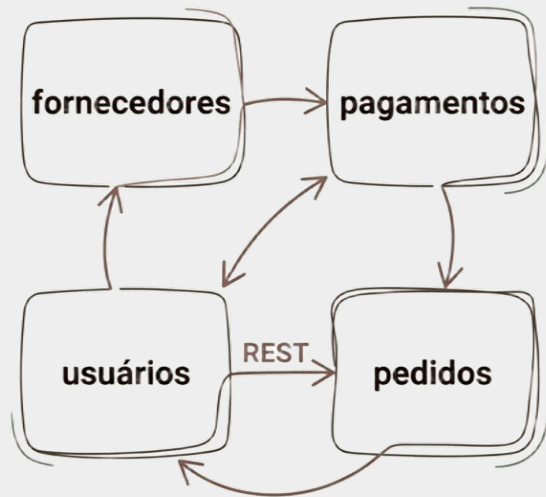
A Decisão que Não Era Arquitetural — Até Ser

A PetFood começou com um monólito pequeno e coeso. Fazia sentido para o contexto. Com o crescimento, cada mudança no módulo de fornecedores exigia um **deploy completo do sistema** — alto risco, alta fricção.

Antes



Depois



❏ A questão nunca foi "monólito vs microsserviços" — foi: **para qual conjunto de trade-offs estamos dispostos?**

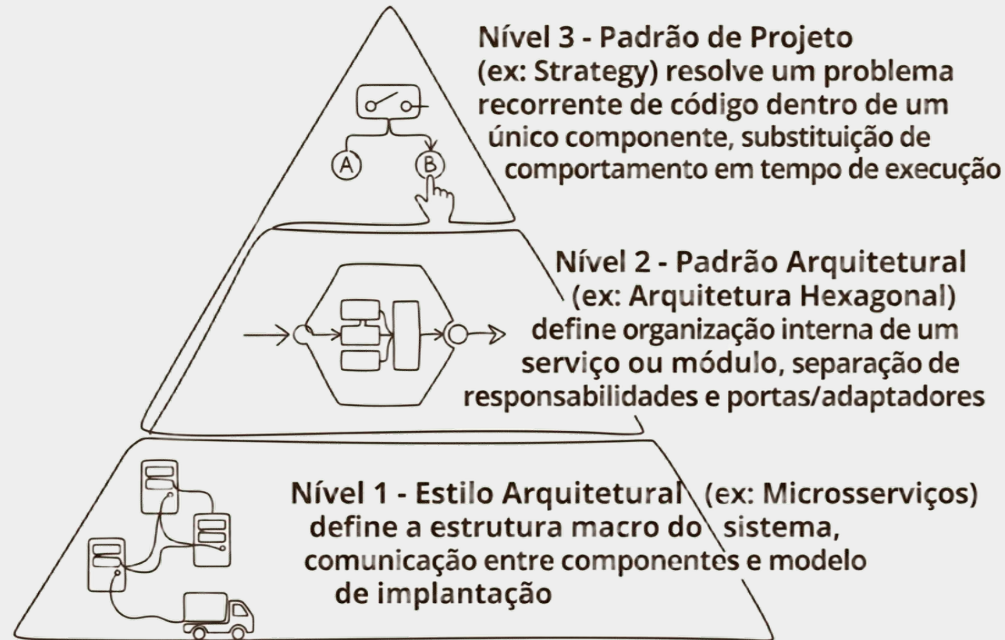
Decisões Arquiteturais se Acumulam

Cada decisão de design cria uma camada sobre as anteriores. Escolhas feitas nos primeiros sprints definem os limites do que é possível construir nos próximos anos, e o custo de reverter essas decisões cresce exponencialmente com o tempo.



Três Níveis de Abstração

Estilo arquitetural, padrão arquitetural e padrão de projeto operam em escalas diferentes, confundi-los leva a decisões de escopo errado e a soluções aplicadas no lugar equivocado.



Estilo Arquitetural: A Forma Macro do Sistema

O que define

O estilo arquitetural determina **como o sistema inteiro é organizado**: seus grandes blocos, os canais de comunicação entre eles e o modelo de implantação.

Exemplos: Microsserviços, Monólito Modular, Event-Driven, SOA.

Implicações práticas

- Define se os times podem fazer deploy independente
- Determina a latência de rede entre componentes
- Estabelece a fronteira de falha e resiliência
- Influencia a estrutura organizacional dos squads (Lei de Conway)



Padrão Arquitetural: A Organização Interna

Enquanto o estilo define o sistema todo, o padrão arquitetural organiza **o interior de cada serviço ou módulo**. A Arquitetura Hexagonal (Ports & Adapters) é um exemplo canônico.



Portas (Ports)

Interfaces que definem o contrato do domínio — independentes de tecnologia. O domínio não sabe se quem chama é uma API REST, um consumer Kafka ou um teste unitário.



Adaptadores (Adapters)

Implementações concretas das portas.
Um adaptador para banco de dados pode ser trocado por outro (ex: PostgreSQL → DynamoDB) sem alterar uma linha do domínio.



Núcleo de Domínio

A lógica de negócio pura, completamente isolada de frameworks, bancos de dados e protocolos. Testável de forma isolada, portátil e duradoura.



Padrão de Projeto: A Solução de Código

Strategy Pattern — um exemplo clássico

O padrão de projeto opera dentro de um único componente, resolvendo um problema recorrente de implementação. O **Strategy** permite trocar algoritmos ou comportamentos em tempo de execução, sem alterar o código cliente.

No contexto da PetFood: o cálculo de frete pode ter estratégias diferentes por transportadora — cada uma implementa a mesma interface `FreteCalculator`, e o carrinho de compras nem sabe qual está ativa.

Onde cada um

Microserviços

Sistema inteiro

Hexagonal

Um serviço

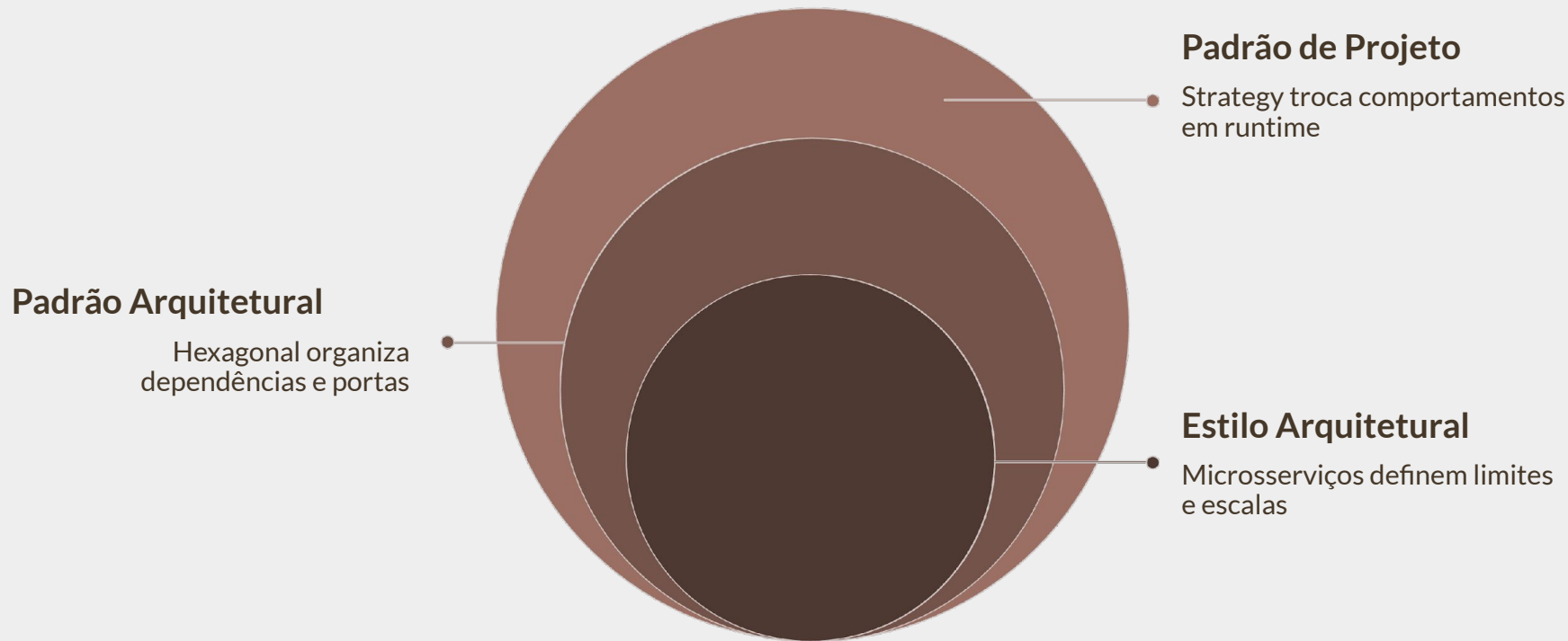
Strategy

Uma classe



Como os Três se Relacionam

Eles são **complementares, não concorrentes**. Um sistema bem projetado usa os três níveis de forma coerente — e a confusão entre eles é uma das fontes mais comuns de decisões de escopo errado.



A metáfora: o estilo é o zoneamento urbano, o padrão arquitetural é a planta do edifício, e o padrão de projeto é o layout do apartamento. Cada decisão vive no nível certo.



SOLID Não É Lista de Boas Práticas — É Diagnóstico

Os princípios SOLID são mais úteis quando usados ao contrário: dado um sintoma no código, qual princípio está sendo violado? Olhe para o seu projeto e tente identificar padrões abaixo.

Princípio violado	Sintoma que você já viu
Single Responsibility	Um componente faz tudo — mexeu em um lugar, quebrou em outro
Open/Closed	Adicionar feature = modificar código existente = PRs enormes
Dependency Inversion	Não dá pra testar sem subir banco de dados real
Interface Segregation	Componente A muda por causa de B, sem razão aparente

❓ Qual violação sua equipe consegue identificar hoje no projeto?



Toda Decisão Arquitetural É um Trade-off

Não existe arquitetura perfeita — existe a arquitetura **mais adequada ao problema**. A PetFood escolheu microsserviços não porque é "melhor", mas porque o trade-off fazia sentido para o contexto deles.

Monólito

- ✓ Simples de desenvolver e testar localmente
- ✓ Sem overhead de rede entre componentes
- ✗ Escala o sistema inteiro, não só a parte necessária
- ✗ Risco crescente de acoplamento alto

Microsserviços

- ✓ Escalável horizontalmente por serviço
- ✓ Times trabalham em paralelo com autonomia
- ✗ Overhead de comunicação e de deploy
- ✗ Testes de integração muito mais difíceis

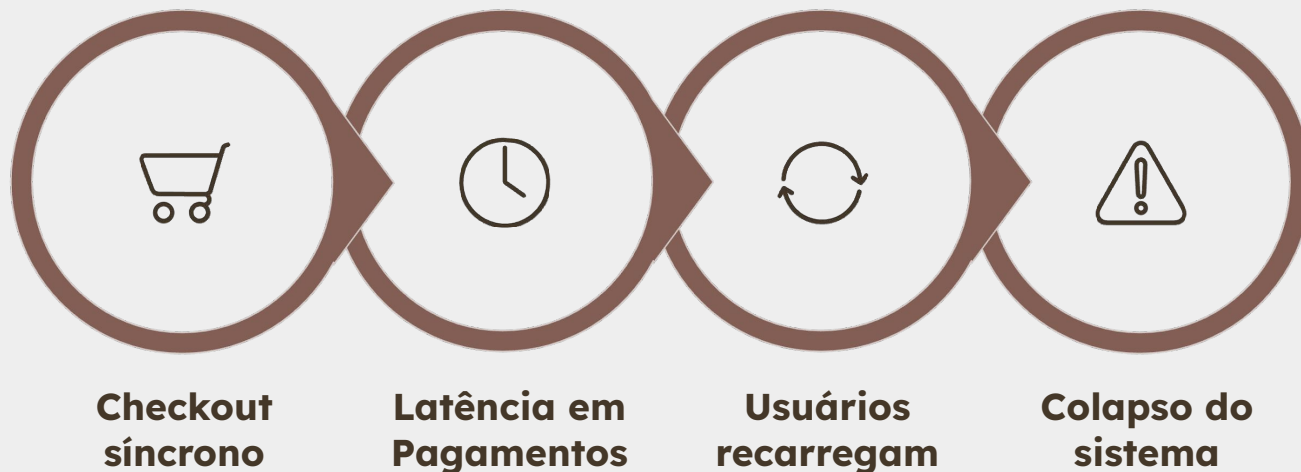
O que faz sentido para o **problema da sua equipe?**



O Dia em que a Falha em Cascata Derrubou o Checkout

ESTUDO DE CASO • PETFOOD

O serviço de pagamentos começou a ter latência. O checkout fazia chamada **síncrona** para pagamentos — e propagou a lentidão para o usuário. Usuários atualizavam a página, gerando mais chamadas, mais carga e, por fim, colapso total.



A "linha" no diagrama de arquitetura era, na prática, um **ponto de acoplamento frágil**. A rede não é confiável. Chamadas síncronas entre componentes críticos são risco arquitetural documentado.



O server.js de 800 Linhas

Uma equipe decidiu na Semana 6 que o backend seria um único server.js com ~800 linhas — "mais simples para começar". Na Semana 10, precisaram adicionar autenticação. Lógica de negócio, acesso ao banco e rotas estavam misturados.

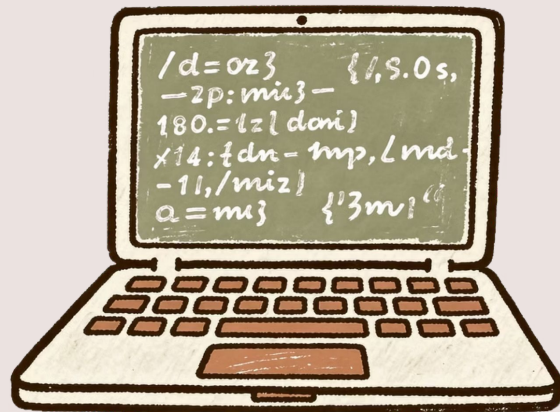
⚠ Resultado: 3 dias de trabalho e 4 bugs novos para implementar uma única funcionalidade.

1. Qual princípio SOLID foi violado?

2. Qual atributo de qualidade foi mais impactado?

3. O que poderia ter sido feito na Semana 6?

4. Qual é a relação com dívida técnica?



1. Qual princípio SOLID foi violado?

1. Principalmente o Single Responsibility Principle (SRP)
 - a. o `server.js` acumula rotas, lógica de negócio e acesso ao banco no mesmo lugar
2. Em segundo lugar, o Dependency Inversion Principle (DIP)
 - a. a lógica de alto nível (regras de negócio) depende diretamente de detalhes de infraestrutura (o banco), sem camada de abstração entre eles



2. Qual atributo de qualidade foi mais impactado?

Manutenibilidade: 3 dias para uma funcionalidade que, num sistema bem estruturado, seria um middleware de algumas linhas

Testabilidade: não há como testar a lógica de negócio sem subir o servidor inteiro com o banco real, porque tudo está acoplado



3. O que poderia ter sido feito na Semana 6?

Separação em camadas desde o início, mesmo que simples, mesmo que "pareça overhead para um projeto pequeno":

<code>routes/</code>	→ só mapeamento de endpoints HTTP
<code>controllers/</code>	→ orquestra a requisição e a resposta
<code>services/</code>	→ regras de negócio (não sabe que é Express)
<code>repositories/</code>	→ acesso ao banco (não sabe que é Express)
<code>middleware/</code>	→ autenticação, validação, logging

Com essa estrutura, autenticação seria adicionada como um middleware em `middleware/auth.js` aplicado às rotas protegidas, sem tocar em nenhuma lógica existente.



4. Qual é a relação com dívida técnica?

A dívida foi **contraída conscientemente**, a equipe sabia que "tudo em um arquivo" era uma simplificação, mas escolheu não pagar o custo da separação na Semana 6.

O problema não foi contrair a dívida; foi não ter plano de pagamento.

Dívida técnica com plano é estratégia. Dívida técnica sem plano é acidente esperando para acontecer.



Como a Separação em Camadas Teria Ajudado

Uma arquitetura em camadas bem definida não é burocracia — é a diferença entre adicionar autenticação em 30 minutos ou em 3 dias. Cada camada tem uma responsabilidade única e não conhece os detalhes das outras.



routes/

Só roteamento HTTP — mapeia URL para controller, nada mais



controllers/

Orquestra a lógica da requisição — recebe, valida, delega e responde



repositories/

Acesso ao banco — sem saber que existe Express ou regra de negócio



services/

Regras de negócio puras — sem saber que existe HTTP ou Express

✔ Autenticação vai em um **middleware**, não espalhada pelo código. Testes unitários testam **services/** sem precisar subir o Express ou o banco.



Por Que Documentar Decisões Arquiteturais?

Em times ágeis, o ritmo de entrega é alto e a documentação frequentemente fica para depois, que nunca chega. O resultado: decisões críticas vivem apenas na memória de quem estava na reunião.



O Problema da Memória Institucional

Rotatividade de times

Em squads de longa duração como os da PetFood, o time que tomou uma decisão em 2021 pode não ser o mesmo time que vai mantê-la em 2024. Sem registro, o contexto se perde.

A armadilha da "boa ideia"

Um novo engenheiro encontra uma decisão que parece arbitrária — ex: "por que não usamos JPA aqui?" — e a reverte sem saber que ela foi tomada após um incidente de performance grave.

Decisões invisíveis

A ausência de documentação não significa ausência de decisão. Significa que a decisão foi tomada implicitamente, e implicitamente pode ser desfeita, gerando inconsistência sistêmica.



Architectural Decision Records (ADRs)

Um ADR é um documento curto e estruturado que registra **uma decisão arquitetural significativa**, junto com o contexto que levou a ela e as consequências esperadas.

Título Nome curto e descritivo da decisão	Contexto O problema e as forças em jogo no momento	Decisão O que foi decidido e por quê
Consequências Trade-offs aceitos, dívidas técnicas assumidas		Status Proposta / Aceita / Obsoleta / Substituída



ADRs no Contexto Ágil da PetFood

Por que ADRs funcionam bem em times ágeis?

Times ágeis tomam decisões continuamente, a cada sprint, a cada refinamento. ADRs não são burocracia: são o **changelog da arquitetura**, versionado junto ao código.

Armazenados no repositório (pasta `/docs/adr/`), são revisados em pull requests como qualquer outro artefato técnico.

Benefícios mensuráveis

- Onboarding mais rápido: novos membros entendem o "porquê" das escolhas
- Debates mais objetivos: a decisão anterior tem contexto registrado
- Governança distribuída: o time é responsável pelo registro, não apenas os arquitetos
- Rastreabilidade: é possível correlacionar uma dívida técnica com a ADR que a originou



Exemplo de ADR — PetFood Suppliers

ⓘ ADR-012: Adoção de Kafka para eventos de pedido entre Suppliers e Order Service

Status: Aceita · **Data:** 2023-03-15 · **Time:** Squad Fornecedores

Contexto

O Order Service chamava o Supplier Service de forma síncrona via REST. Em picos de Black Friday, a latência do fornecedor derrubava o checkout. A equipe precisava de desacoplamento temporal.

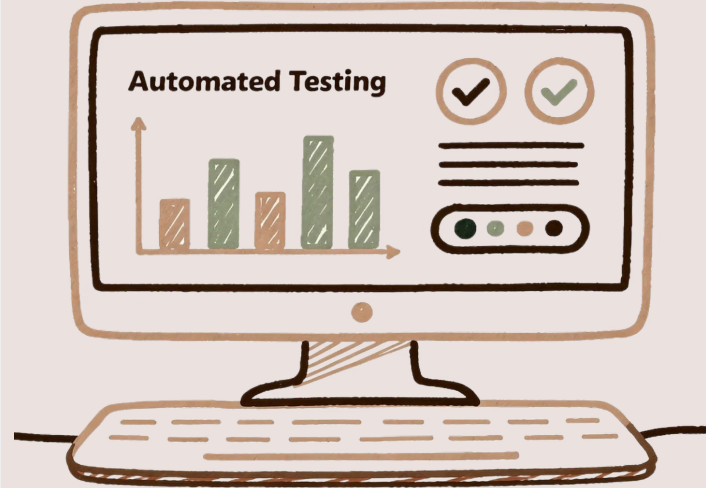
Consequências aceitas

- Consistência eventual (não mais imediata)
- Necessidade de idempotência nos consumidores
- Complexidade operacional do cluster Kafka
- Ganho: resiliência e throughput no checkout



Fitness Functions

O que seria de uma decisão arquitetural se ninguém verificasse se ela ainda está sendo respeitada? As Fitness Functions transformam restrições arquiteturais em testes automatizados que rodam a cada build.



O Problema da Governança Reativa

Cenário reativo (sem Fitness Functions)

A arquitetura hexagonal foi decidida em 2022. Em 2024, durante uma revisão de código, um engenheiro descobre que um adapter está chamando diretamente a camada de domínio com lógica de banco de dados. A violação existe há 8 meses. Ninguém percebeu.

Cenário proativo (com Fitness Functions)

Um teste ArchUnit verifica, a cada pull request, que nenhuma classe do pacote `adapter` importa classes do pacote `domain.model` diretamente. A violação é bloqueada antes do merge. O time é notificado imediatamente.



O Que São Fitness Functions?

Termo popularizado por Neal Ford, Rebecca Parsons e Pat Kua em *Building Evolutionary Architectures*. Uma Fitness Function é qualquer mecanismo que avalia se a arquitetura ainda satisfaz seus objetivos — de forma objetiva e automatizada.



Dependências de Camada

ArchUnit, jqAssistant: verificam que as camadas do sistema só comunicam com as camadas permitidas, prevenindo acoplamento accidental.



Segurança e Compliance

OWASP Dependency-Check, Snyk: verificam que nenhuma dependência com CVE crítico entra na base de código.



Performance e SLAs

Testes de carga no pipeline de CI: garantem que nenhum commit introduz regressão de latência acima do threshold definido.



Coesão e Acoplamento

Métricas como instabilidade e abstração de pacotes: alertam quando um módulo está se tornando um "God Module" antes que o problema seja irreversível.



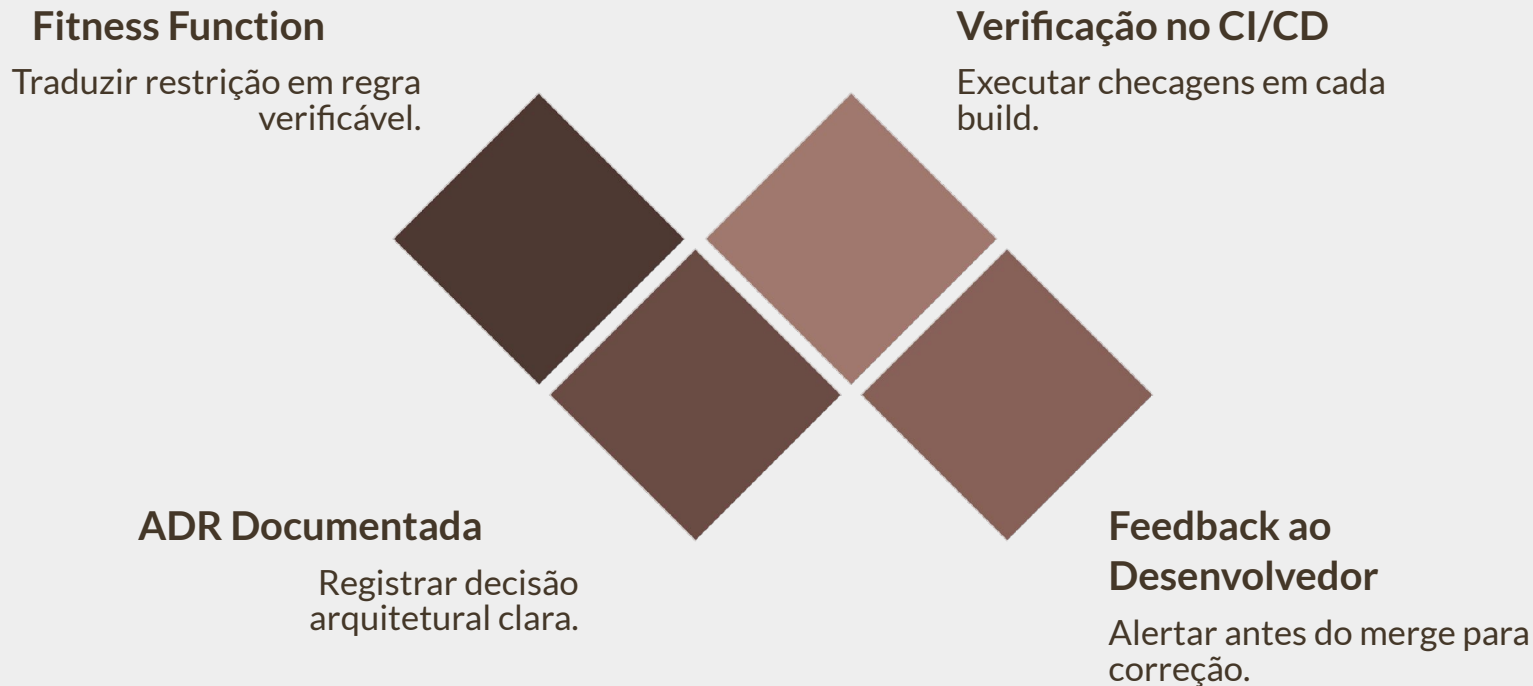
Fitness Functions na Prática

Exemplos concretos que uma equipe como os squads da PetFood pode implementar hoje, com ferramentas open source.

Objetivo	Ferramenta	O que verifica
Isolamento do domínio	ArchUnit (Java)	Domínio não importa adapters
Latência de API	Gatling + CI	P99 < 300ms no endpoint de pedido
Cobertura mínima	JaCoCo + Maven	Build falha se cobertura < 80%
Vulnerabilidades	Snyk / Dependabot	Bloqueia CVE crítico no merge
Tamanho de serviço	Script customizado	Alerta se serviço ultrapassa 30 endpoints



De Reativa a Proativa: A Transformação

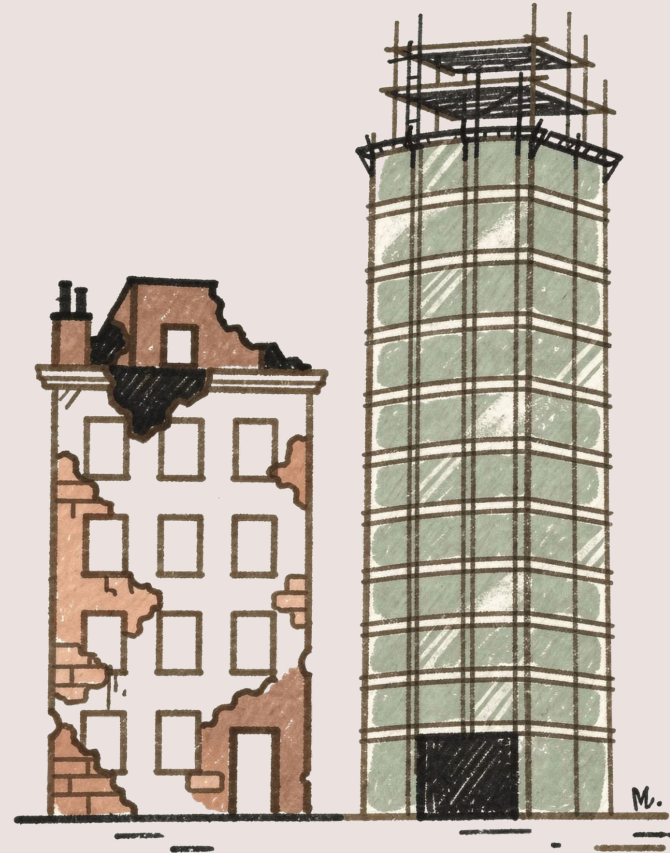


A Fitness Function fecha o ciclo: a decisão arquitetural deixa de ser uma promessa e passa a ser uma **invariante verificada continuamente**. A governança torna-se parte natural do fluxo de desenvolvimento, não uma auditoria periódica.



Modernizando o Monólito de Fornecedores

O antigo sistema de gestão de fornecedores da PetFood: um monólito de 8 anos, acoplado, com cobertura de testes próxima de zero. A equipe precisa decidir como modernizá-lo. Duas abordagens se apresentam.



M.



Strangler Fig Pattern

A metáfora

A figueira estranguladora cresce ao redor de uma árvore existente, gradualmente substituindo-a sem derrubá-la de uma vez. No software: novas funcionalidades são construídas no novo sistema enquanto o legado continua servindo os fluxos existentes.

Como funciona na prática

- Um **proxy/fachada** (ex: API Gateway) roteia requisições entre legado e novo sistema
- Fluxos são migrados incrementalmente, um por um
- O legado é "estrangulado" até que possa ser desligado com segurança
- Rollback possível a qualquer momento durante a transição



Reescrita "Big Bang"

⚠ A reescrita total é frequentemente chamada de "a coisa mais arriscada que uma empresa de software pode fazer" — Joel Spolsky, 2000. Essa afirmação continua relevante em 2024.

O que é

A equipe para o desenvolvimento de novas funcionalidades no legado, reescreve o sistema do zero em paralelo, e faz o *cutover* em uma única janela de manutenção planejada.

Por que é arriscado

- O sistema legado carrega lógica de negócio implícita não documentada
- Meses de desenvolvimento sem entregar valor ao negócio
- O legado continua acumulando demandas durante a reescrita
- Risco de escopo crescente — "só mais essa funcionalidade"

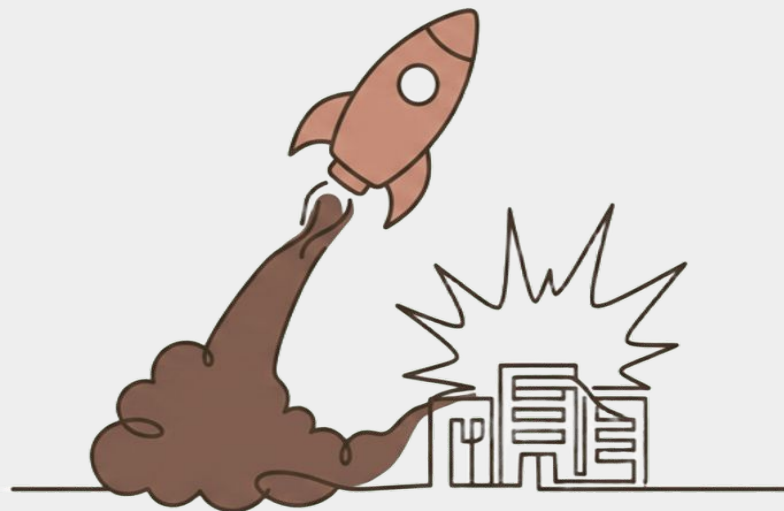


Comparação: Strangler Fig vs. Big Bang



Padrão Strangler Fig

- Migração incremental
- Baixo Risco
- Entrega Contínua de Valor
- Rollback Possível



Reescrita Big Bang

- Migração Total
- Alto Risco
- Sem Entrega durante Reescrita
- Sem Rollback Fácil



Fatores Técnicos na Decisão



Cobertura de Testes

Cobertura baixa favorece Strangler Fig, sem testes, uma reescrita não tem como validar fidelidade comportamental. Cada rota migrada pode ser testada isoladamente.



Acoplamento e Coesão

Se o monólito é um "Big Ball of Mud" com ciclos de dependência, o Strangler Fig pode ser difícil de aplicar. Reescritas pontuais de módulos coesos podem ser preferíveis.



Compartilhamento de Banco

Monólitos com banco compartilhado exigem estratégias adicionais (ex: Strangler + Database Decomposition). O anti-padrão "Integration Database" é o maior obstáculo técnico à migração incremental.



Fatores de Negócio na Decisão



Velocidade de

Entrega

Quanto tempo pode tolerar meses sem novas features enquanto a reescrita ocorre? Geralmente não, o Strangler Fig entrega valor continuamente durante a migração.



Custo de Oportunidade

Quanto custa manter o legado rodando em paralelo? Se a infraestrutura do monólito é cara demais para manter durante a migração, a pressão pelo Big Bang aumenta.



Conhecimento de Negócio

Regras de negócio implícitas no legado são o maior risco do Big Bang. Se não há especialistas que conhecem o sistema profundamente, a reescrita total tende a omitir comportamentos críticos.



Janela de Mercado

Se existe uma janela de mercado que exige um sistema novo em produção em 3 meses, a dinâmica muda. Mas essa pressão raramente justifica o risco — e frequentemente leva a estimativas irreais.



O Caso PetFood: Qual Escolher?

Aplicando os critérios ao monólito de fornecedores da PetFood, o Strangler Fig Pattern emerge como a escolha mais segura — mas com uma ressalva importante.

Recomendação: Strangler Fig

- Sistema crítico com 8 anos de lógica implícita
- Cobertura de testes próxima de zero
- Time de negócio não pode pausar entregas
- Risco de rollback inaceitável para operações

Pré-condições necessárias

- Mapear e priorizar os bounded contexts do domínio
- Implementar o proxy de roteamento como primeiro passo
- Aumentar cobertura do legado antes de migrar cada rota
- Definir critérios de "done" para cada fluxo migrado





Estudo de Caso: O Efeito Cascata de uma Decisão

Um exemplo real de como uma única decisão arquitetural tomada sem ADR, sem Fitness Function e sem o padrão correto gerou uma cascata de consequências, e como a equipe da PetFood a resolveu.



A Decisão Inicial: Banco Compartilhado

2019 — Sprint 1

Time decide que Order Service e Supplier Service compartilham o mesmo schema PostgreSQL para "simplificar" joins de relatórios. Não há ADR.

2021 — Patch

Time adiciona cache Redis como paliativo. Agora há inconsistência eventual não intencional, e ninguém documentou as garantias de consistência esperadas.

2020 — Escala

Volume de pedidos cresce 5x. Locks de tabela causam contenção. A tentativa de escalar o Order Service falha, o banco é o gargalo compartilhado.

2023 — Crise

Migração para microsserviços bloqueada. O "banco compartilhado" está sendo usado por 7 serviços. Strangler Fig não pode avançar sem resolver o acoplamento de dados primeiro.



O Que Poderia Ter Sido Diferente

Com as práticas do Cap. 14 aplicadas desde o início, cada ponto de falha do estudo de caso poderia ter sido evitado — ou detectado antes de se tornar crise.

→ **ADR no Sprint 1**

Documentar "escolhemos banco compartilhado por simplicidade operacional, trade-off: impossibilidade de escalar serviços independentemente" forçaria a equipe a confrontar o trade-off explicitamente antes de aceita-lo.

→ **Fitness Function para isolamento de dados**

Um teste que verificasse que nenhum serviço acessa tabelas de outro serviço diretamente detectaria a violação antes que 7 serviços fossem afetados.

→ **Padrão Hexagonal no Supplier Service**

Com adaptadores de banco isolados, a migração do banco do Supplier para um schema próprio seria uma troca de adapter, não uma refatoração de meses.



Conexão com as Decisões da Semana 6

As escolhas que seu squad fez na Semana 6 já estão criando camadas. Avaliar essas decisões à luz das ferramentas do Cap. 14 é o exercício desta semana.

Qual estilo arquitetural vocês escolheram?

Monólito modular,
microserviços, event-driven?
O estilo está documentado?
Os trade-offs foram
explicitados?

Existe um ADR para a decisão mais impactante?

Identifique a decisão que, se revertida hoje, causaria mais impacto. Ela está documentada com contexto e consequências?

Qual Fitness Function vocês implementariam primeiro?

Se pudessem automatizar apenas uma verificação arquitetural agora, qual seria?
Por que essa e não outra?

Se houvesse um legado, qual abordagem usariam?

Dado o sistema que vocês projetaram, se ele envelhecesse 5 anos sem manutenção, seria candidato ao Strangler Fig ou ao Big Bang?



Síntese: Decisões de Design e Qualidade

Qualidade não é uma propriedade emergente, é o resultado acumulado de centenas de decisões conscientes, documentadas e verificadas continuamente.



Estilo, Padrão, Design

Cada nível de abstração resolve um problema diferente. Aplicar no nível errado gera soluções de escopo equivocado.



ADRs

O changelog da arquitetura. Contexto preservado, decisões rastreáveis, onboarding acelerado.



Fitness Functions

Governança automatizada. A decisão vira invariante verificada a cada build, não uma promessa verbal.



Strangler Fig

Modernização incremental e segura. Para sistemas críticos, a evolução gradual supera o risco da reescrita total.



Mapeamento de Decisões Arquiteturais

Estudo de caso: decisão arquitetural e consequências: 'As decisões da Semana 6, que impacto tiveram?'

Cada equipe preenche a tabela abaixo para suas próprias decisões arquiteturais.

Decisão tomada (quando)	Atributo afetado	Impacto observado	Boa decisão?
...	...	...	...
...	...	...	...
...	...	...	...

 Não existe resposta certa ou errada, o objetivo é **nomear e refletir**, não julgar.



Isso Que Estamos Fazendo Tem Nome: Retrospectiva Técnica

Profissionais seniores analisam decisões regularmente — nomeiam impactos, documentam o que fariam diferente. O artefato que registra isso formalmente é o **ADR (Architecture Decision Record)**.

O que decidimos

Descrição objetiva da decisão tomada e seu contexto

Por quê decidimos assim

A motivação e os fatores que tornaram essa opção a mais adequada no momento

Alternativas consideradas

Outras opções que foram avaliadas e os motivos pelos quais foram descartadas

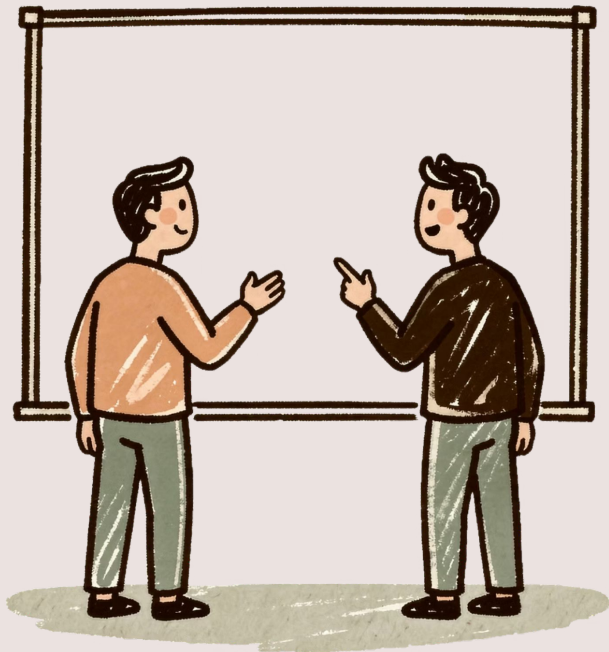
Trade-offs aceitos

O que foi sacrificado conscientemente para viabilizar a decisão tomada

A PetFood usa ADRs para cada decisão arquitetural significativa.



Uma Pergunta Sem Resposta Certa



Escolha uma das perguntas abaixo para reflexão e debate. 2 min individual → 5 min debate → síntese do professor.

Opção A

"Como os princípios arquiteturais que estudamos poderiam ter mudado uma decisão concreta que sua equipe tomou nos últimos três meses?"

Opção B

"Em que momento uma 'boa decisão de arquitetura' se torna uma 'má decisão de arquitetura' — e quem é responsável por identificar essa transição?"



O Stakeholder Vai Experimentar as Consequências Dessas Decisões

Na quinta-feira acontece a **Sprint 3 Review** com o stakeholder. Ele não vai avaliar "a arquitetura" diretamente — mas vai sentir o resultado dela em cada interação com o produto.



O produto responde rápido?

Decisões de desempenho e acoplamento determinam a experiência percebida pelo usuário



Tem bugs nos fluxos principais?

Baixa testabilidade e alta coesão indevida aumentam a densidade de defeitos



Funcionalidades foram entregues?

Dívida técnica acumulada reduz a velocidade de entrega nas sprints finais

⚠ A qualidade que você construiu (ou não) nas últimas semanas é o que ele vai experimentar na quinta.



Para Amanhã — Terça-feira

Amanhã é uma aula totalmente prática: **debugging sistemático com breakpoints no VS Code** e preparação final para a Sprint Review. Sem leitura prévia necessária.

O que trazer

- Notebook com o projeto rodando localmente
- Um bug ou comportamento inesperado que sua equipe não conseguiu explicar ainda

Referências desta aula

- Garcia, V. C. *Engenharia de Software em Dimensões* — Cap. 14, seções 14.7–14.8
- Valente, M. T. *Engenharia de Software Moderna* — Cap. 5 e Cap. 7



Boa Sprint Review.

Que as decisões que vocês tomaram nas próximas semanas sejam mais conscientes do que as das semanas passadas, esse é o objetivo da aula de hoje.



Resumo dos Conceitos da Semana 11

01

Decisão arquitetural

Estruturalmente importante, difícil de reverter, impacto duradouro

02

Atributos de qualidade

Manutenibilidade, desempenho, segurança e testabilidade são afetados pela arquitetura

03

Trade-offs conscientes

Não existe bala de prata — existe a escolha mais adequada ao contexto

04

SOLID como diagnóstico

Use os princípios para identificar problemas, não apenas para justificar boas práticas

05

ADR — Architecture Decision Record

Artefato profissional para documentar decisões, alternativas e trade-offs aceitos



Referências e Recursos

Garcia, V. C.

Engenharia de Software em Dimensões

Capítulo 14 — seções 14.7 e 14.8

Arquitetura, componentes e decisões estruturais

Valente, M. T.

Engenharia de Software Moderna

Capítulo 5 — Princípios de Design

Capítulo 7 — Arquitetura

Leitura Complementar

Documentação de ADRs: adr.github.io

Fowler, M. — *Patterns of Enterprise Application Architecture*

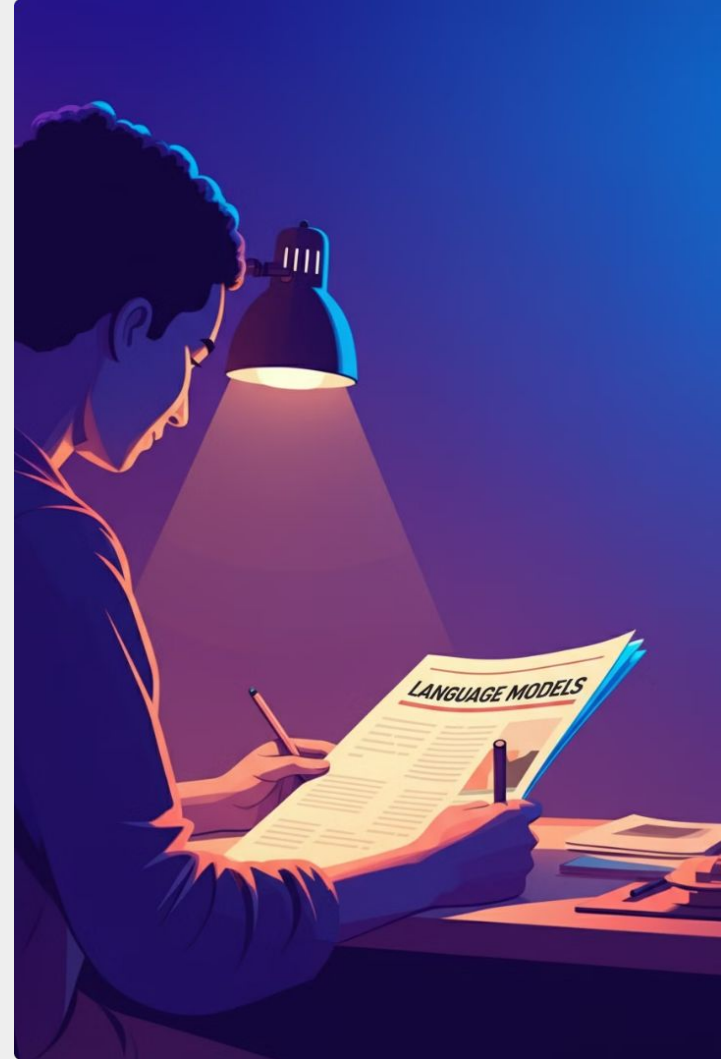
Conceitos de acoplamento e coesão

CIN0136 — Desenvolvimento de Software • CIn-UFPE • 2026.1 • Semana 11



Próximos passos

- Eng. Software em Dimensões
 - Cap. 14 (seções 14.1–14.8):
Arquitetura, C4 Model, qualidade arquitetural
- Eng. Software Moderna
 - "Cap. 5 — Princípios de Projeto Coesão, acoplamento, SOLID
 - Cap. 7 — Arquitetura Camadas, MVC, trade-offs"



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Impacto das Decisões de Design
na Qualidade